Alejo Almada

DOCUMENTACIÓN TÉCNICA · v1.0 · 2026

LENGUAJES IV · TRABAJO FINAL INTEGRADOR

Manual de Diseño, Arquitectura y Código Fuente

Cómo está pensado, construido y organizado el Portfolio de Desarrollador Fullstack: capas, base de datos, seguridad, API, módulos del código, pruebas y despliegue.

React 19 · Tailwind CSS v4 · Node.js · Supabase · PostgreSQL 17 · Row Level Security · Vercel

Integrantes

Sanchez Fadell, Francisco
Cadima, Francisco
Arraya, Joaquín
Almada, Alejo
Juárez, Agustín

Sitio publicado

proyecto-final-lenguajes-iv.vercel.app

Versión

Promoción (rol Administrador)

CONTENIDO

Índice

01	Introducción y alcance	Objetivo, roles, equipo
02	Manual de diseño	UX/UI, sistema de diseño, responsive
03	Arquitectura	Capas, despliegue, decisiones
04	Estructura del proyecto	Carpetas y archivos
05	Modelo de datos	Tablas, relaciones, métricas
06	Seguridad	Roles, RLS, validaciones
07	Flujos principales	Registro, contacto, imágenes, métricas
08	API REST	Endpoints
09	Código fuente	Servidor, base de datos, cliente
10	Pruebas, puesta en marcha y límites	npm test, scripts, Vercel

01 · INTRODUCCIÓN

Qué es el sistema

Un **portfolio de desarrollador fullstack** con tres caras: un **sitio público** (perfil, stack, servicios, experiencia, proyectos, actividad de GitHub, clima y formulario de contacto), un **panel del operador** (bandeja de consultas, seguimiento por estados y respuesta por correo) y un **panel del administrador** (CRUD de proyectos, demos y servicios, gestión de usuarios, contadores de visitas y accesos, y gráficos con consultas parametrizadas). El detalle de los proyectos y sus demos es sólo para usuarios registrados.

ROL	CÓMO SE OBTIENE	QUÉ PUEDE HACER
Visitante anónimo	—	Ver la portada y las tarjetas de proyectos; enviar consultas
Visitante registrado	Se registra en `/ingresar` (email o Google)	Lo anterior + detalle y demos de proyectos
Operador	Lo asigna un administrador	Lo anterior + bandeja de consultas
Administrador	Script `admin:crear` o el panel	Todo el panel

Reparto del trabajo

ÁREA	INTEGRANTE	ARCHIVOS PRINCIPALES
Diseño y UI	Juárez, Agustín	`docs/manual-de-marca/` · `client/src/index.css` · `componentes/`
Frontend interactivo	Almada, Alejo	`secciones/` · `paginas/` · `contexto.jsx` · `supabase.js` · `animaciones.js`
Backend y seguridad	Cadima, Francisco	`server/src/http.js` · `permisos.js` · `validar.js` · `rutas/` · `imagenes.js`
Datos, RLS y métricas	Arraya, Joaquín	`supabase/migrations/` · `metricas.js` · `consultas.js` · `panel/Metricas.jsx`
Integraciones, calidad y despliegue	Sanchez Fadell, Francisco	`correo.js` · `externas.js` · `server/test/` · `vercel.json`

Diseño UX/UI

La identidad visual completa (paleta #HEX, tipografía, componentes, wireframes) está en el documento aparte **Manual de Marca y Guía de Estilo**. Acá se resumen las decisiones de diseño que afectan al código.

SISTEMA DE DISEÑO EN CÓDIGO

Los colores, fuentes, radios y breakpoints del manual son **tokens de Tailwind CSS v4** en el bloque `@theme` de `index.css`. El manual y el código usan los mismos nombres (`bg-surface-2` , `text-primary` ...).

COMPONENTES BASE

`ui.jsx` define `Boton` , `Campo` , `Entrada` , `Seleccion` , `Insignia` , `Tarjeta` , `Paginacion` ... Toda pantalla se arma con ellos, así el estilo es uniforme.

RESPONSIVE MOBILE-FIRST

Breakpoints `sm` 576 · `md` 768 · `lg` 992 · `xl` 1200 px. Grillas que pasan a 1 columna, menú lateral, tablas con scroll horizontal y botones ≥ 44 px.

ACCESIBILIDAD

Contraste ≥ 4,5:1, foco visible, `label` en todo campo, errores con `role="alert"` , ventanas que atrapan el foco y respeto de `prefers-reduced-motion` .

MOVIMIENTO

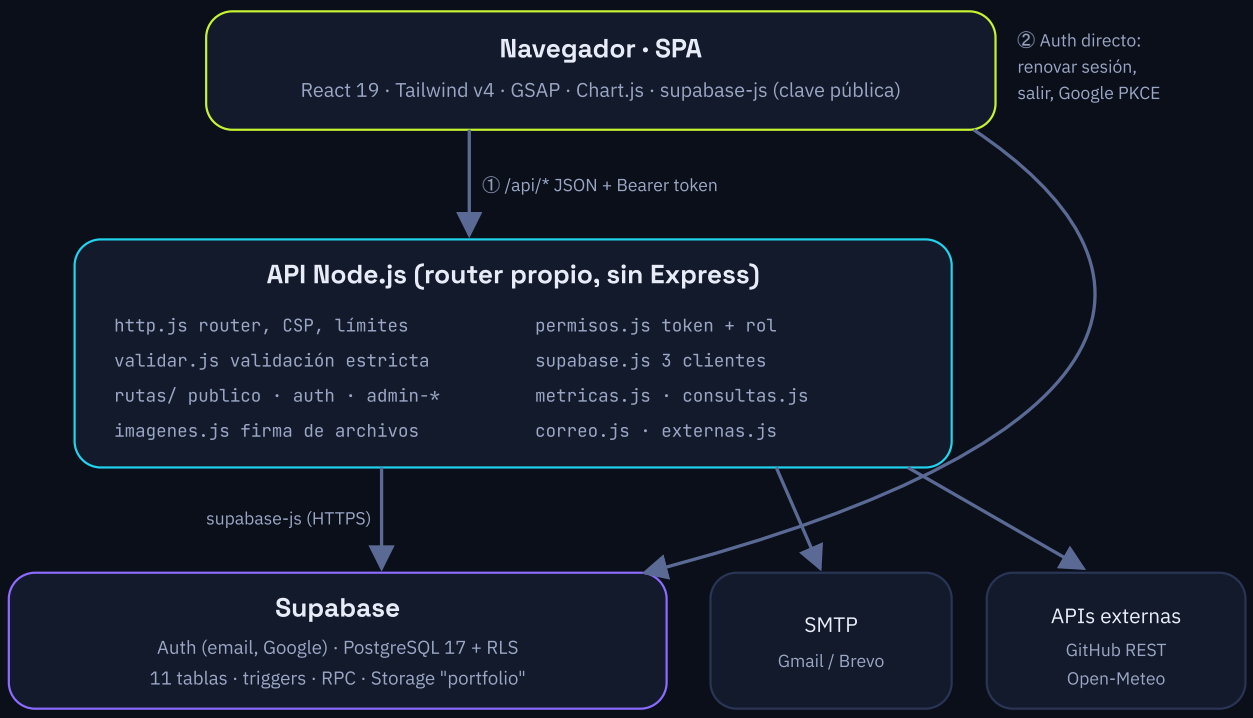
GSAP + ScrollTrigger en `animaciones.js` : línea de tiempo del hero, parallax, apariciones escalonadas y contadores.

MENSAJES

Todo error se muestra en español y dice cómo arreglarlo. El servidor genera los textos (`validar.js`) y el cliente los muestra tal cual.

Vista general

Arquitectura en **tres capas**: un cliente React (SPA), una API Node.js propia y **Supabase** como backend de datos (PostgreSQL, Auth y Storage). La API agrega lo que Supabase no hace solo: validación con mensajes claros, límites anti-abuso, correos SMTP, APIs externas cacheadas y reglas de negocio.



Principios

- **Doble control de permisos:** el servidor verifica el rol en cada ruta (`soloStaff` , `soloAdmin`) *y* casi todas las consultas se hacen con el token del propio usuario, así la base vuelve a verificar con **Row Level Security**.
- **Mínimo privilegio:** la clave `service_role` nunca sale del servidor; se usa sólo para guardar consultas del formulario, contar visitas y administrar cuentas de Auth después de comprobar que quien lo pide es admin.
- **Validación en tres capas:** navegador, servidor y restricciones `CHECK` de la base.

Despliegue

El sitio está publicado en **Vercel**: `proyecto-final-lenguajes-iv.vercel.app`. Cada `git push` a `main` lo vuelve a publicar.

SITIO ESTÁTICO

Vite compila `client/` a `client/dist`; la CDN de Vercel lo sirve. Toda ruta que no es `/api` ni `/assets` devuelve `index.html` (SPA).

API SERVERLESS

`api/index.js` es una función de Vercel que reutiliza **el mismo código** del servidor local (`crearManejador` + `crearRutas`).

DATOS

Supabase en la nube (proyecto `portfolio-lenguajes4`) con las migraciones de `supabase/migrations/` aplicadas en orden.

`api/index.js` — la API completa como función serverless

```
const config = require('../server/src/config');
config.validar();
const { crearManejador } = require('../server/src/http');
const crearRutas = require('../server/src/app');
const atender = crearManejador(crearRutas());
module.exports = (req, res) => atender(req, res);
```

`vercel.json` agrega las cabeceras de seguridad: `Content-Security-Policy` (`script-src 'self'`, `connect-src` sólo propio + Supabase, `frame-ancestors 'none'`), `Strict-Transport-Security`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff` y caché inmutable para `/assets`. También puede correr en **localhost** o en un VPS (`deploy/Caddyfile` + `portfolio.service`).

Decisiones de diseño

DECISIÓN	MOTIVO
Supabase (DB + Auth + Storage)	Base administrada, Auth probado (bcrypt, refresh tokens, OAuth), Storage con CDN y RLS para defensa en profundidad.
API Node **delante** de Supabase	Validación con mensajes en español, límites anti-abuso, correos y reglas de negocio en un solo lugar.
Cliente "del usuario" en vez de `service_role`	La base aplica RLS con la identidad real de quien pide.
Funciones SQL para métricas	Consultas parametrizadas tipadas, con validación y control de rol dentro de la base.
Columnas generadas `busqueda` y `prioridad`	Buscar sin acentos y ordenar la bandeja sin armar SQL a mano.
Router HTTP propio (sin Express)	Cero dependencias en el servidor salvo `supabase-js` y `nodemailer`; control total de cabeceras y límites.
Panel con `React.lazy`	Chart.js sólo se descarga al entrar a `/panel`.

Carpetas y archivos

Monorepo con el cliente, el servidor y la base de datos separados. Unas 8.400 líneas de código propio (JS, JSX, SQL y CSS).

```
proyecto final/
├── api/index.js                función serverless de Vercel (toda la API)
├── client/                       React + Vite + Tailwind
│   ├── index.html · vite.config.js · public/favicon.svg
│   └── src/
│       ├── main.jsx · App.jsx · router.jsx   arranque y rutas de la SPA
│       ├── index.css                         TOKENS del manual de marca (@theme)
│       ├── supabase.js                       cliente Supabase del navegador (sesión, Google)
│       ├── api.js                            todas las llamadas a /api (adjunta el token)
│       ├── contexto.jsx                      sesión, datos del sitio y avisos
│       ├── animaciones.js · utils.js          GSAP · validación del lado del navegador
│       ├── componentes/  ui · Layout · Superposicion · Icono
│       ├── secciones/    Hero · Stack · Servicios · Experiencia · Proyectos · ActividadGithub · Contacto
│       ├── paginas/      Inicio · ProyectoDetalle · Ingresar · AuthCallback · NoEncontrada
│       └── panel/        Panel · Consultas · Metricas · Grafico · GestionProyectos ·
│                         GestionServicios · Usuarios · SubirImagen · compartido
├── server/                         API Node.js ≥ 20.12
│   ├── src/
│   │   ├── index.js · app.js · config.js     arranque, registro de rutas, variables
│   │   ├── http.js · errores.js · seguridad.js
│   │   ├── permisos.js · supabase.js · validar.js
│   │   ├── consultas.js · metricas.js · imagenes.js · correo.js · externas.js
│   │   └── rutas/    publico · auth · proyectos · admin-consultas ·
│   │                 admin-contenido · admin-metricas · admin-usuarios
│   ├── scripts/  seed · crear-admin · probar-correo · generar-seed-sql
│   └── test/     unit · db (migraciones + RLS en PGlite) · integracion (Supabase real)
├── supabase/
│   ├── migrations/   6 migraciones SQL (aplicar en orden)
│   └── seed.sql                contenido de ejemplo
├── scripts/      preparar.js (arma todo) · dev.js (servidor + sitio)
├── deploy/       Caddyfile · portfolio.service
├── docs/         arquitectura · supabase · pruebas · auditoría · requisitos · manual de marca
└── vercel.json · package.json · README.md
```

Base de datos relacional

PostgreSQL 17 administrado por Supabase. 11 tablas en el esquema `public`, todas con RLS activado, más `auth.users` (Supabase Auth) y `storage.objects` (imágenes).

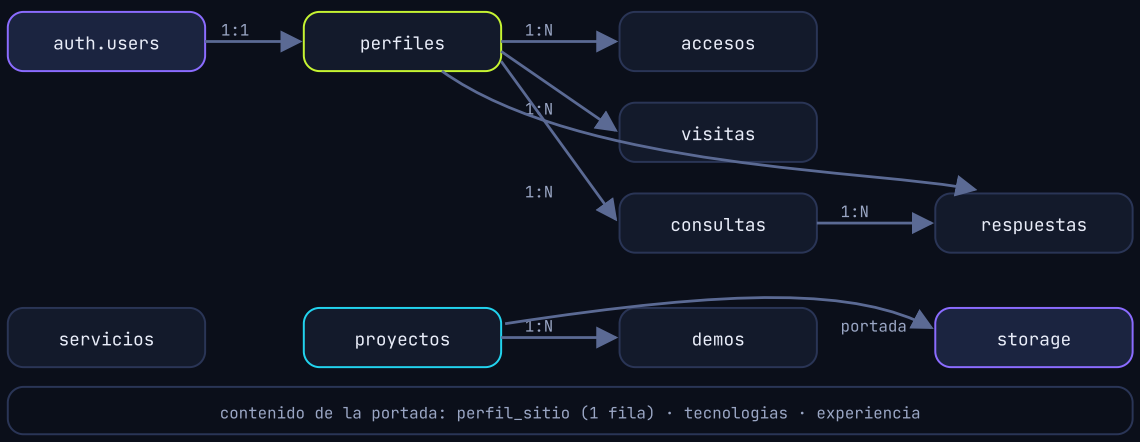


TABLA	PARA QUÉ	RESTRICCIONES IMPORTANTES
perfiles	Rol (`admin` / `operador` / `visitante`), nombre, activo, proveedor de cada cuenta de Auth	FK a `auth.users` en cascada; el trigger `al_crear_usuario` lo crea **siempre como visitante**; `proteger_perfiles` impide quedarse sin admins
perfil_sitio, tecnologias, experiencia	Contenido de la portada	`CHECK` de largos, URLs `https` , categorías cerradas
servicios	Servicios ofrecidos (CRUD)	Ícono de una lista cerrada, orden 0–9999
proyectos, demos	Portfolio (CRUD)	Slug con formato, URLs sólo `https` , máx. 12 tecnologías, demos en cascada
consultas	Formulario de contacto	Tipo y estado en listas cerradas; columnas generadas `busqueda` (sin acentos) y `prioridad`
respuestas	Correos enviados por el operador	`simulado` si no había SMTP; sólo staff firma a su nombre
visitas	Contador de tráfico	Hash diario de 16 hex (sin IP); `usuario_id` si había sesión
accesos	Accesos autenticados	**Única (usuario, sesión)**: la misma sesión no cuenta dos veces

Definición de las métricas

- **Visita** = una sesión de navegación (una por pestaña; `registrar_visita` ignora otra del mismo visitante en 30 min; se descartan bots).
- **Visitante único** = hash diario de IP + navegador + secreto (no se guarda la IP). **Acceso autenticado** = una sesión nueva de Supabase Auth; recargar no suma.

Roles y permisos

ACCIÓN	ANÓNIMO	REGISTRADO	OPERADOR	ADMIN	DÓNDE SE VERIFICA
Portada, servicios, tarjetas	✓	✓	✓	✓	RLS `*_ver` + `proyectos_publicos()`
Detalle de proyecto y demos	✕ 401	✓	✓	✓	`requiereUsuario` + RLS
Enviar consulta	✓*	✓	✓	✓	Validación + límite → service_role
Bandeja, estados, responder	✕	✕	✓	✓	`soloStaff` + RLS `es_staff()`
CRUD proyectos, demos, servicios	✕	✕	✕	✓	`soloAdmin` + RLS `es_admin()` + Storage
Usuarios y métricas	✕	✕	✕	✓	`soloAdmin` + `exigir_admin()`

* Con límite de 5 envíos cada 15 minutos y campo trampa contra bots.

Row Level Security

Toda tabla tiene RLS. El rol se lee **en la base** con funciones `SECURITY DEFINER`; nunca lo decide el navegador. Así, aunque alguien use la clave pública directo contra la API de Supabase, la tabla `proyectos` le devuelve `[]`.

`supabase/migrations/…_seguridad.sql`

```
create or replace function public.es_admin() returns boolean
language sql stable security definer set search_path = ''
as $$ select coalesce(public.rol_actual() = 'admin', false); $$;

-- Sólo usuarios autenticados ven proyectos (anon no tiene política SELECT)
create policy proyectos_ver on public.proyectos for select to authenticated
  using (activo or (select public.es_admin()));
create policy proyectos_insertar on public.proyectos for insert to authenticated
  with check ((select public.es_admin()));
```

Otras medidas

- Contraseñas y sesiones de **Supabase Auth** (bcrypt, refresh tokens revocables, PKCE para Google). Una cuenta desactivada pierde el acceso al instante.
- Límite de intentos por IP y por email en el ingreso; límite de envíos en el formulario.
- CSP estricta, enlaces de demos y repositorios sólo `https`, iframes con `sandbox`.
- Un trigger garantiza que siempre quede al menos un administrador activo.

Validación en tres capas

El enunciado pide validación estricta de todos los campos, archivos e imágenes. Cada dato se controla tres veces: si una capa falla o se saltea, la siguiente lo frena.

CAPA	CAMPOS	ARCHIVOS E IMÁGENES
1 · Navegador	`client/src/utils.js`: formato, largos y obligatorios antes de enviar; error junto al campo	`SubirImagen.jsx`: tipo, peso y redimensionado a 1280 px con Canvas
2 · Servidor	`server/src/validar.js`: tipos, largos, email, URL `https`, slug, listas cerradas, fechas; limpia caracteres de control	`imagenes.js`: tipo declarado **y primeros bytes** del archivo, máx. 4 MB, nombre generado por el servidor
3 · Base	Restricciones `CHECK`, `UNIQUE` y FK en la migración de esquema	Bucket con límite de 4 MB y tipos MIME; RLS: sólo un admin sube o borra

`server/src/validar.js` — validación de texto

```
function texto(valor, campo, { max = 255, obligatorio = false, min = 0, multilinea = false } = {}) {
  if (valor === undefined || valor === null) valor = '';
  if (typeof valor !== 'string') throw invalido(`${campo}: el valor no es válido.`);
  let limpio = valor.replace(/\r\n/g, '\n');
  limpio = multilinea
    ? limpio.replace(/[\u0000-\u0008\u000b\u000c\u000e-\u001f\u007f]/g, '')
    : limpio.replace(/[\u0000-\u001f\u007f]/g, ' ');
  limpio = limpio.trim();
  if (obligatorio && !limpio) throw invalido(`${campo} es obligatorio.`);
  if (limpio.length > max) throw invalido(`${campo} no puede superar los ${max} caracteres.`);
  return limpio;
}
```

`server/src/imagenes.js` — se verifica la firma real del archivo, no la extensión

```
function firmaValida(datos, ext) {
  if (ext === 'jpg')  return datos[0] === 0xff && datos[1] === 0xd8 && datos[2] === 0xff;
  if (ext === 'png')  return datos.subarray(0, 8).equals(Buffer.from([0x89, 0x50, 0x4e, 0x47, 0x0d, 0x0a,
0x1a, 0x0a]));
  if (ext === 'webp') return datos.subarray(0, 4).toString('latin1') === 'RIFF'
                          && datos.subarray(8, 12).toString('latin1') === 'WEBP';
  return false;
}
```

Cómo viajan los datos

REGISTRO E INGRESO CON EMAIL

`Ingresar.jsx` → `POST /api/auth/registro|login` (el servidor valida y limita intentos por IP y email) → Supabase Auth crea o valida la cuenta → el trigger crea el perfil como *visitante* → el servidor registra el acceso con el token del usuario → devuelve `{ sesion, usuario }` → el navegador guarda la sesión con `supabase.auth.setSession`, que se renueva sola.

INGRESO CON GOOGLE

`supabase.auth.signInWithOAuth` (PKCE) → Google → `/auth/callback?code=…` → `AuthCallback.jsx` canjea el código → `POST /api/auth/acceso` cuenta el acceso → redirige a donde estaba.

CONSULTA → RESPUESTA

Formulario → `POST /api/contacto` (validación + campo trampa + 5 envíos/15 min) → se guarda con código `CON-XXXXXX` → correo al dueño y confirmación al visitante → el operador la ve en la bandeja y cambia su estado (*Nueva* → *En seguimiento* → *Respondida* → *Archivada*, con notas fechadas) → responde → **si el SMTP falla no se guarda nada ni cambia el estado** (502); si sale, se guarda en `respuestas` y pasa a *Respondida*.

IMAGEN DE UN PROYECTO

El navegador valida tipo y peso y la achica → `POST /api/admin/imagenes` → el servidor verifica los primeros bytes (máx. 4 MB) → la sube a Storage con el token del admin (RLS) → devuelve la URL pública. Al cambiar o borrar el proyecto, la imagen anterior se elimina del bucket.

MÉTRICAS

`Metricas.jsx` (rango de fechas, agrupación día/semana/mes, tipo de consulta) → `/api/admin/metricas/*` → RPC `metricas_resumen`, `metricas_series` y `metricas_consultas` con **parámetros tipados** → la función valida el rango y el rol dentro de la base → Chart.js dibuja los gráficos interactivos.

Cerrar sesión: `supabase.auth.signOut()` revoca el refresh token; el access token deja de servir al vencer (≤ 1 h) y además el servidor rechaza al instante una cuenta desactivada.

Endpoints

JSON sobre HTTPS. Las rutas protegidas reciben `Authorization: Bearer <token de Supabase>`. Errores: 400 validación · 401 sin sesión · 403 sin permiso · 404 · 409 conflicto · 415 tipo de archivo · 429 límite · 502 correo.

PÚBLICO · SESIÓN

GET	/api/sitio
GET	/api/portfolio
POST	/api/contacto
POST	/api/visitas
GET	/api/github · /api/clima
POST	/api/auth/registro
POST	/api/auth/login
POST	/api/auth/acceso
GET	/api/auth/yo
GET	/api/proyectos/:slug (registrado)

OPERADOR / ADMIN · CONSULTAS

GET	/api/admin/consultas
GET	/api/admin/consultas/resumen
GET	/api/admin/consultas/:id
PATCH	/api/admin/consultas/:id/estado
PATCH	/api/admin/consultas/:id/notas
POST	/api/admin/consultas/:id/responder

ADMIN · MÉTRICAS

GET	/api/admin/metricas/resumen
GET	/api/admin/metricas/series
GET	/api/admin/metricas/consultas

ADMIN · CONTENIDO

GET	/api/admin/servicios[/:id]
POST	/api/admin/servicios
PUT	/api/admin/servicios/:id
DELETE	/api/admin/servicios/:id
GET	/api/admin/proyectos[/:id]
POST	/api/admin/proyectos
PUT	/api/admin/proyectos/:id
DELETE	/api/admin/proyectos/:id
GET	/api/admin/proyectos/:id/demos
POST	/api/admin/proyectos/:id/demos
PUT	/api/admin/demos/:id
DELETE	/api/admin/demos/:id
POST	/api/admin/imagenes

ADMIN · USUARIOS

GET	/api/admin/usuarios
GET	/api/admin/usuarios/:id
POST	/api/admin/usuarios
PATCH	/api/admin/usuarios/:id
DELETE	/api/admin/usuarios/:id

Ejemplo de consulta parametrizada: `GET /api/admin/metricas/series?desde=2026-10-01&hasta=2026-11-09&agrupar=week&tipo=cotizacion`. Un valor como `desde='; DROP TABLE` responde 400.

API Node.js

MÓDULO	LÍNEAS	RESPONSABILIDAD
http.js	333	Mini servidor sin dependencias: router, lectura de JSON con límite de tamaño, archivos estáticos con gzip, CSP y cabeceras de seguridad
permisos.js	83	Valida el token con Supabase Auth, lee rol y estado; middlewares `requiereUsuario`, `usuarioOpcional`, `soloStaff`, `soloAdmin`
supabase.js	72	Tres clientes: `publico`, `deUsuario(token)` (RLS) y `admin` (service_role); traduce errores de Supabase a HTTP
validar.js	169	Validación de texto, enteros, decimales, booleanos, email, URL, slug, listas, fechas y contraseñas
seguridad.js	71	Límites de uso por IP/email y bloqueo por intentos fallidos
consultas.js	211	Crear, listar, filtrar, cambiar estado, notas fechadas y responder consultas
metricas.js	79	Lee y valida parámetros; llama a las funciones RPC de métricas
imagenes.js	67	Subida y borrado de portadas en Supabase Storage
correo.js	144	Plantillas HTML (con escape) y envío SMTP con nodemailer; modo simulado
externas.js	120	GitHub REST y Open-Meteo con caché de 15–30 min
rutas/*.js	~600	Un archivo por área: publico, auth, proyectos, admin-consultas, admin-contenido, admin-metricas, admin-usuarios

server/src/permisos.js — autenticación y doble control

```
async function cargarUsuario(ctx) {
  const cabecera = ctx.req.headers.authorization || '';
  const token = cabecera.startsWith('Bearer ') ? cabecera.slice(7).trim() : '';
  if (!token || token.length > 4096 || token.split('.').length !== 3) return null;
  // 1. Supabase Auth valida firma, vencimiento y que la sesión no esté revocada
  const { data, error } = await supabase.publico.auth.getUser(token);
  if (error || !data?.user) return null;
  // 2. Se lee el perfil con el token del usuario: la base aplica RLS con SU identidad
  const db = supabase.deUsuario(token);
  const { data: perfil } = await db.from('perfiles')
    .select('id, email, nombre, rol, activo, avatar_url').eq('id', data.user.id).maybeSingle();
  if (!perfil || !perfil.activo) return null;   // desactivado = sin acceso al instante
  ctx.db = db;                                  // 3. las rutas consultan con este cliente
  return perfil;
}
```

server/src/rutas/admin-metricas.js — una ruta típica

```
router.get('/api/admin/metricas/series', ...soloAdmin, async (ctx) => {
  const parametros = metricas.leerParametros(ctx.query);   // valida desde/hasta/agrupar/tipo
  return { ...parametros, puntos: await metricas.series(ctx.db, parametros) };
});
```

Migraciones SQL

La base se reconstruye desde cero aplicando las migraciones en orden (`supabase db push` o el SQL Editor). Son re-ejecutables.

MIGRACIÓN	CONTENIDO
…000100_esquema.sql	Tablas, restricciones `CHECK`, columnas generadas e índices
…000200_seguridad.sql	Funciones de rol, triggers (`al_crear_usuario` , `proteger_perfiles`), políticas RLS de las 11 tablas y de Storage
…000300_funciones.sql	RPC: `proyectos_publicos` , `registrar_visita` , `metricas_resumen` , `metricas_series` , `metricas_consultas` , `consultas_resumen` , `usuarios_resumen`
… 000400_endurecer_funciones.sql	Correcciones del Security Advisor de Supabase (permisos de ejecución)
…000500_accesos_propios.sql	Cada usuario puede leer sus accesos (necesario para el `RETURNING` de PostgREST)
…000600_foto_perfil.sql	Foto de perfil del sitio guardada en el bucket

supabase/migrations/…_funciones.sql — consulta parametrizada de métricas

```sql
create or replace function public.metricas_series(
  p_desde date, p_hasta date,
  p_agrupar text default 'day',
  p_zona text default 'America/Argentina/Salta',
  p_tipo text default null
)
returns table (periodo text, visitas integer, visitantes integer, visitas_autenticadas integer,
               accesos integer, usuarios_con_acceso integer, registros integer,
               usuarios_acumulados integer, consultas integer)
language plpgsql stable set search_path = ''
as $$
begin
  perform public.exigir_admin();                                  -- rol verificado EN LA BASE
  perform public.validar_rango_metricas(p_desde, p_hasta, p_agrupar);
  if p_tipo is not null and p_tipo not in ('general', 'cotizacion', 'asesoria') then
    raise exception 'Tipo de consulta no válido.' using errcode = '22023';
  end if;
  -- … agrupa visitas, accesos, registros y consultas por período …
end; $$;
```

Los valores llegan como **parámetros tipados** (`date` , `text`) por RPC; nunca se concatenan en una cadena SQL, por eso no hay inyección posible.

React + Tailwind

MÓDULO	RESPONSABILIDAD
router.jsx · App.jsx	Router propio con la History API; desplazamiento a las anclas de la portada; panel cargado con `React.lazy`
contexto.jsx	Contextos de sitio (nombre, Google habilitado, correo simulado), sesión de Supabase Auth y avisos
api.js	Única puerta al servidor: adjunta el token, traduce errores a `ErrorApi` con mensaje listo para mostrar
supabase.js	Cliente de Supabase del navegador (sólo clave pública): sesión persistente y login con Google
componentes/	`ui.jsx` (sistema de componentes), `Layout.jsx` (barra, menú celular, pie), `Superposicion.jsx` (modales accesibles)
secciones/	Bloques de la portada: Hero, Stack, Servicios, Experiencia, Proyectos, ActividadGithub, Contacto
paginas/	Inicio, ProyectoDetalle (con puerta de registro), Ingresar, AuthCallback, NoEncontrada
panel/	Consultas (bandeja), Metricas + Grafico (Chart.js), GestionProyectos, GestionServicios, Usuarios, SubirImagen

client/src/api.js — todas las llamadas pasan por acá

```
async function pedir(metodo, ruta, { cuerpo, crudo, tipo, auth = false, token, signal } = {}) {
  const headers = {};
  const guardado = token || (auth ? await tokenActual() : '');
  if (guardado) headers.Authorization = `Bearer ${guardado}`;
  // … JSON o archivo crudo (imágenes), fetch, y si el servidor responde 401 se cierra la sesión
}
```

client/src/index.css — los tokens del Manual de Marca como tema de Tailwind v4

```
@import "tailwindcss";
@theme {
  --color-surface-1: #0b0f1a;   --color-surface-2: #131a2b;
  --color-ink: #e8ecf8;         --color-primary: #c6f432;
  --color-secondary: #22d3ee;   --color-accent: #8b6cff;
  --font-display: "Space Grotesk", ui-sans-serif, system-ui, sans-serif;
  --font-sans: "IBM Plex Sans", ui-sans-serif, system-ui, sans-serif;
  --breakpoint-md: 768px;  --breakpoint-lg: 992px;
  --radius-md: 12px;  --radius-lg: 20px;
  --shadow-focus: 0 0 0 3px #c6f432;
}
```

client/src/componentes/ui.jsx — variantes del botón con clases del tema

```
const VARIANTES = {
  primario:   'bg-primary text-on-primary hover:bg-primary-hover font-semibold',
  secundario: 'border border-line-strong bg-transparent text-ink hover:border-primary hover:text-
primary',
  peligro:    'bg-danger/15 text-danger border border-danger/40 hover:bg-danger/25',
};
```

10 · PRUEBAS

Pruebas automáticas

UNITARIAS · 14

Validaciones, límites de uso, lectura del token, errores de Supabase, escape HTML en correos, firma de imágenes, parámetros de métricas.

BASE DE DATOS · 16

Ejecutan **las migraciones reales** en PostgreSQL local (PGlite) y prueban RLS como cada rol, triggers, `CHECK`, deduplicación e inyección SQL.

INTEGRACIÓN · 9

Servidor contra Supabase real con cuentas de los 3 roles: CRUD, subida a Storage, bandeja, métricas, alta/bloqueo/baja de usuarios.

`npm test` corre las 30 locales sin internet; `npm run test:integracion` las 9 contra Supabase. Las pruebas encontraron y permitieron corregir dos bugs reales (permiso de `exigir_admin()` y la política SELECT de `accesos`).

10 · PUESTA EN MARCHA

Comandos

COMANDO	QUÉ HACE
`npm run preparar`	Genera `client/.env`, instala dependencias, comprueba la base, carga contenido de ejemplo y crea usuarios de prueba
`npm run dev`	Enciende servidor y sitio en `http://localhost:5173`
`npm run construir · npm run iniciar`	Versión de producción en un solo puerto (`:3000`)
`npm run admin:crear -- email "Nombre" "clave" [rol]`	Crea un administrador u operador
`npm run correo:probar --prefix server -- tu@mail.com`	Prueba la configuración SMTP

Requisitos: Node.js ≥ 20.12 y un proyecto de Supabase con las migraciones aplicadas. Las claves van en `server/.env` (la `service_role` sólo ahí); el navegador recibe sólo datos públicos.

Límites conocidos

- Las demos en `iframe` dependen de que el sitio destino permita mostrarse incrustado; por eso hay "Abrir en otra pestaña".
- Los límites de uso del servidor viven en memoria: se reinician y no se comparten entre instancias (Supabase Auth tiene además los suyos).
- No hay pantalla propia de recuperación de contraseña; el admin puede cambiarla desde Usuarios.
- Perfil, tecnologías y experiencia se editan desde el Table Editor de Supabase (el enunciado pide ABM de proyectos, demos y servicios).